

Hafta 1 — Detay ve Derin Referans



Bu dosya sınav kapsamı DEĞİLDİR.

Buradaki hiçbir konu vize veya finalde sorulmaz. Bu dosya, **merak eden ve projesinde daha ileri gitmek isteyen** öğrenci içindir. Sınava `H01-ders-notu.md` dosyasından çalışın.

İçindekiler

1. Sunucu tarafı webin kısa tarihi
2. ASP.NET WebForms neden bırakıldı
3. Ham HTTP istek/yanıt örnekleri (derste kullanılır)
4. Kestrel, IIS ve ters vekil
5. HTTP/1.1, HTTP/2, HTTP/3
6. Diğer sunucu çatıları — karşılaştırma
7. `CreateBuilder` arka planda ne yapar
8. Yapılandırma sağlayıcıları ve öncelik sırası
9. `MapStaticAssets` ile `UseStaticFiles` farkı
10. Kaynakça

1. Sunucu Tarafı Webin Kısa Tarihi

Webin ilk yıllarında sunucular yalnızca **dosya sunuyordu**. Dinamik içerik ihtiyacı doğunca birbirini izleyen çözümler geldi:

Dönem	Yaklaşım	Nasıl çalışırdı	Sorunu
1990'ların başı	CGI (Common Gateway Interface)	Her istek için ayrı bir işletim sistemi süreci başlatılır, çıktısı tarayıcıya gönderilir	Her istekte süreç başlatmak çok pahalı; yüzlerce eşzamanlı kullanıcıda çöker
1990'ların ortası	Sunucu modülleri (mod_perl, ISAPI)	Kod, web sunucusunun içinde çalışır	Sunucuya sıkı bağımlılık, taşınabilirlik yok
1996	ASP (Active Server Pages)	HTML içine gömülü VBScript	Kod ve görünüm iç içe; büyük projede yönetilemez
2002	ASP.NET WebForms	Masaüstü benzeri bileşen modeli	\$2
2009	ASP.NET MVC	Sorumlulukları ayıran desen	Yalnızca Windows/IIS
2016	ASP.NET Core	Çapraz platform, modüler, açık kaynak	—

Bugüne kalan fikir: CGI'in "her istek için süreç" modeli öldü, ama "istek gelir → kod çalışır → yanıt üretilir" fikri değişmedi. Bugün yazdığınız controller action'ı, kavramsal olarak bir CGI betiğinin yaptığı işi yapıyor — yalnızca çok daha ucuza.

2. ASP.NET WebForms Neden Bırakıldı

WebForms (2002), masaüstü programcılarını web'e çekmek için tasarlandı. Sayfaya bir düğme sürüklüyor, `Button1_Click` metodunu yazıyordunuz — tıpkı bir Windows uygulamasındaki gibi.

Bunu mümkün kılmak için **webin doğasını gizlemesi** gerekti:

- **ViewState** : Sayfanın tüm durumu, gizli bir form alanında **Base64 kodlanmış** olarak istemciye gönderilir ve her gönderimde geri gelirdi. Karmaşık sayfalarda bu alan yüzlerce kilobayta ulaşabiliyordu — her istekte, iki yönde.
- **PostBack** : Her etkileşim, sayfanın kendisine bir `POST` isteği idi. URL değişmezdi.
- **Sunucu kontrolleri**: Üretilen HTML'in üzerinde denetiminiz yoktu; `id` değerleri `ctl00$ContentPlaceHolder1$TextBox1` gibi otomatik üretiliyordu.

Neden bırakıldı:

1. **Soyutlama sızdırıyordu.** HTTP durumsuzdur; WebForms durumu varmış gibi davranıyordu. Bir şey ters gittiğinde, altta gerçekte ne olduğunu bilmeden hata ayıklayamıyordunuz.
2. **Test edilemiyordu.** Sayfa yaşam döngüsü ile kod iç içeydi.
3. **Modern istemci tarafıyla çatışıyordu.** Üretilen HTML ve `id` 'ler üzerinde denetim olmadan JavaScript/CSS yazmak zordu.
4. **Yalnızca Windows'ta çalışıyordu.**

✦ **Sınavla ilgili not:** Sınav sorularındaki `GridView`, `PostBack`, `.aspx` gibi seçenekler **çeldiricidir** — WebForms kavramlarıdır ve bu derste öğretilmez. Doğru cevap hiçbir zaman bunlar değildir.

3. Ham HTTP İstek/Yanıt Örnekleri

✦ Bu bölüm, **Uygulama A'nın internetsiz yedek planıdır** (`H01-uygulama.md`). İnternet yoksa aşağıdaki metinler yansıtılıp aynı sorular sorulur.

3.1. Bir `GET` isteği ve yanıtı

İstek:

```
GET /Urun/Detay/42 HTTP/1.1
Host: kampuskitap.local
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: text/html,application/xhtml+xml,application/xml;q=0.9
Accept-Language: tr-TR,tr;q=0.9
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Cookie: .AspNetCore.Session=CfDJ8NrAkS...
```

(Son satırdaki boşluk bilinçlidir: **GET** isteğinin gövdesi yoktur, başlıklardan sonra gelen boş satır isteğin bittiğini gösterir.)

Yanıt:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 3417
Date: Mon, 16 Feb 2027 08:14:22 GMT
Server: Kestrel
Cache-Control: no-cache, no-store

<!DOCTYPE html>
<html lang="tr">
<head><title>Veri Yapıları – KampusKitap</title></head>
<body>
  <h1>Veri Yapıları ve Algoritmalar</h1>
  <p>Fiyat: 120 TL</p>
</body>
</html>
```

Sorulacaklar: Metot? Durum kodu? Sunucu hangi tipte veri döndüğünü nasıl söylüyor? İstemci kim olduğunu hangi satırda söylüyor?

3.2. Bir **POST** isteği ve yönlendirme yanıtı

İstek:

```
POST /Uye/Giris HTTP/1.1
Host: kampuskitap.local
Content-Type: application/x-www-form-urlencoded
Content-Length: 41

eposta=ayse@example.com&parola=gizli123
```

Yanıt:

```
HTTP/1.1 302 Found
Location: /Uye/Profil
Set-Cookie: .AspNetCore.Identity.Application=CfDJ8...; path=/; httponly; samesite=lax; secure
Content-Length: 0
```

Vurgulanacak üç nokta:

1. **POST** 'un **gövdesi vardır** ve **Content-Length** onun uzunluğudur.
2. **302** + **Location** = "*işini yaptım, şimdi şu adrese git*". Tarayıcı bu adrese otomatik olarak yeni bir **GET** isteği yapar. Bu, **Hafta 4**'teki **PRG deseninin** ta kendisidir.
3. **Set-Cookie**, stateless sorununun çözümünün başladığı satırdır (Hafta 7). **httponly** = JavaScript okuyamaz, **secure** = yalnızca HTTPS'te gider.

3.3. Bir hata yanıtı

```
HTTP/1.1 404 Not Found
Content-Type: text/html; charset=utf-8
Content-Length: 1245
```

404, isteğin **işlendiğini** ama kaynağın bulunamadığını gösterir. Sunucu çökmemiştir — bu önemli bir ayrımdır. Çökseydi **500** görürdünüz.

4. Kestrel, IIS ve Ters Vekil

ASP.NET Core uygulaması çalıştığında içinde **Kestrel** adlı bir web sunucusu vardır. Kestrel çapraz platformdur ve çok hızlıdır.

Peki neden hâlâ IIS veya Nginx duyuyorsunuz? Çünkü üretimde genellikle **iki katmanlı** bir yapı kullanılır:



Neden doğrudan Kestrel'i internete açmıyoruz? Açabilirsiniz — Kestrel bunu destekler. Ama ters vekil şunları ücretsiz verir: sertifika yönetimi, birden çok uygulamayı tek porttan sunma, yavaş istemci saldırılarına karşı koruma, uygulama çökerse yeniden başlatma.

✦ **Sınır:** Bu konunun devamı ([Bilişim Sistemleri Altyapı ve Teknolojileri](#) , [Sistem Yöneticiliği](#)) dersleridir. Bu derste Hafta 13'te yalnızca tanıma düzeyinde geçilir.

5. HTTP/1.1, HTTP/2, HTTP/3

	HTTP/1.1	HTTP/2	HTTP/3
Standart	1997	RFC 7540 (2015)	RFC 9114 (2022)
Bişim	Düz metin	İkili (binary)	İkili
Taşıma	TCP	TCP	QUIC (UDP üzerinde)
Aynı anda çok istek	Sıra beklenir	Çoğullama (multiplexing)	Çoğullama
Başlık sıkıştırma	Yok	HPACK	QPACK

Neden önemli: HTTP/1.1'de bir bağlantı üzerinden aynı anda tek istek gidebildiği için tarayıcılar aynı sunucuya 6 bağlantı açardı. HTTP/2 bunu çözdü: **tek bağlantı, çok istek**.

HTTP/3, TCP'yi bırakıp QUIC'e geçerek "head-of-line blocking" sorununu taşıma katmanında da çözer — bir paket kaybolduğunda diğer akışlar beklemez.

Kodunuzu etkiler mi? Neredeyse hiç. ASP.NET Core bunu sizin yerinize halleder; yazdığınız controller aynı kalır. Etkilediği yer **performans optimizasyonlarıdır**: HTTP/2'de "dosyaları birleştirme" (bundling) eski bir alışkanlık hâline gelmiştir.

6. Diğer Sunucu Çatıları

Aynı sorunu çözen farklı ekosistemler:

Çatı	Dil	MVC'ye karşılığı	Ayırt edici özelliği
ASP.NET Core	C#	Controller + View + Model	Güçlü tiplendirme, entegre DI, yüksek performans
Spring Boot	Java	@Controller + Thymeleaf	Kurumsal dünyada çok yaygın; yoğun yapılandırma
Django	Python	View + Template + Model	"Piller dahil": hazır yönetim paneli, ORM
Laravel	PHP	Controller + Blade + Eloquent	Hızlı başlangıç, paylaşımlı barındırmada kolay
Express	JavaScript	Elle kurulur	Minimal; her şeyi siz seçersiniz
Ruby on Rails	Ruby	Controller + ERB + ActiveRecord	Sözleşme odaklılığın kaynağı

Ortak olan ne? Hepsinde şunlar vardır: yönlendirme, Middleware Pipeline, şablon motoru, ORM, oturum yönetimi, form doğrulama. Bu dersi bitiren biri, Django belgelerini açtığında kavramların karşılıklarını tanır. **Öğrendiğiniz şey ASP.NET Core değil, sunucu tarafı web programlamadır** — ASP.NET Core yalnızca onu öğrendiğimiz araçtır.

7. CreateBuilder Arka Planda Ne Yapar

`WebApplication.CreateBuilder(args)` tek satırdır ama arkasında epeyce iş vardır:

- Yapılandırmayı kurar** — `appsettings.json`, ortam dosyası, ortam değişkenleri, komut satırı argümanları (§8'deki sırayla)
- Günlüklemeyi (logging) kurar** — konsol ve hata ayıklama sağlayıcıları
- Dependency Injection kabını hazırlar** — `IServiceCollection`
- Kestrel'i yapılandırır** — dinlenecek adres ve portlar
- Ortamı belirler** — `ASPNETCORE_ENVIRONMENT` değişkeninden (`Development` / `Production`)
- Temel servisleri kaydeder** — `IConfiguration`, `ILogger<T>`, `IWebHostEnvironment` gibi

Yani `builder.Services.AddControllersWithViews()` yazmadan önce bile, kabınızda onlarca servis kayıtlıdır. Hafta 7'de DI'ı işlerken buna döneceğiz.

8. Yapılandırma Sağlayıcıları ve Öncelik Sırası

`CreateBuilder` varsayılan olarak şu kaynakları **bu sırayla** okur. **Sonraki, öncekini ezer**:

1. appsettings.json
2. appsettings.{Environment}.json (örn. appsettings.Development.json)
3. Kullanıcı sırları (user secrets) (yalnızca Development ortamında)
4. Ortam değişkenleri
5. Komut satırı argümanları

Pratik sonucu: `appsettings.json` içinde `"Sehir": "Sakarya"` yazıyorsa ve uygulamayı şöyle çalıştırırsanız:

```
dotnet run --Sehir=Ankara
```

Değer **Ankara** olur. Komut satırı en yüksek önceliklidir.

Neden böyle tasarlandı: Aynı derlenmiş uygulamayı farklı ortamlara değiştirmeden dağıtabilmek için. Üretimdeki veritabanı parolası koda veya `appsettings.json` 'a değil, **ortam değişkenine** konur.

⚠ **Sırlar** `appsettings.json` 'a **yazılmaz**. O dosya kod deposuna gider. Geliştirme sırasında `dotnet user-secrets`, üretimde ortam değişkeni veya bir sır kasası kullanılır. Hafta 7 ve Hafta 13'te buna döneceğiz.

9. MapStaticAssets ile UseStaticFiles Farkı

.NET 9'dan itibaren MVC şablonu `app.UseStaticFiles()` yerine `app.MapStaticAssets()` üretir. Ders notunda şablonun ürettiği biçimi öğrendiniz; ikisinin farkı şudur:

	UseStaticFiles()	MapStaticAssets()
Ne zaman geldi	Baştan beri	.NET 9
Nasıl çalışır	Çalışma zamanında diskten okur	Derleme/yayın zamanında bilinen varlıkları hazırlar
Sıkıştırma	Yok (elle yapılandırılır)	Otomatik (gzip; yayında brotli de)
Önbellek etiketi (ETag)	Dosya tarihine göre	İçeriğin SHA-256 özetine göre
Diskten/gömülü kaynaktan sunma	✅ Yapabilir	❌ Yapamaz

Ne zaman hangisi: Uygulamanızın `wwwroot` 'unda duran normal varlıklar için `MapStaticAssets` daha iyidir — şablon bu yüzden onu koyar. Ancak dosyaları **çalışma zamanında** başka bir yerden sunacaksınız (kullanıcıların yüklediği dosyalar gibi) `UseStaticFiles` gerekir.

Bunun sebebi, `MapStaticAssets` 'in çalışma zamanında `wwwroot` 'u taramak yerine **derleme ve yayın zamanında üretilen bir varlık bildiriminden (manifest)** okumasıdır. Bildirimde olmayan dosya bu yolla sunulamaz.

✦ Hafta 13'te dosya yükleme yaparken bu ayırım tekrar karşımıza çıkacak: kullanıcının yüklediği dosyalar derleme zamanında bilinmediği için `MapStaticAssets` onları kapsamaz.

Kaynak (doğrulandı, 18 Eylül 2026): Bu bölümdeki üç iddia da resmî dokümantasyonda açıkça yazılıdır — derleme zamanı sıkıştırma ("*Gzip ... is used during development. Gzip and Brotli ... are both used during publish*"), SHA-256 tabanlı ETag ("*a Base64-encoded string of the SHA-256 hash of each file's content*") ve disk/gömülü kaynak sınırı ("*If the app serves assets from other locations, such as disk or embedded resources, use static file middleware*"). Bkz. <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/static-files?view=aspnetcore-10.0>

10. Kaynakça

Resmî dokümantasyon (hepsi `?view=aspnetcore-10.0` ile açılmalıdır)

- ASP.NET Core'a genel bakış — <https://learn.microsoft.com/en-us/aspnet/core/>
- MVC başlangıç öğreticisi (10 bölüm) — <https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-mvc-app/start-mvc?view=aspnetcore-10.0>
- Yapılandırma — <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/configuration/?view=aspnetcore-10.0>
- Statik dosyalar — <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/static-files?view=aspnetcore-10.0>

Sürüm ve destek

- .NET destek politikası — <https://dotnet.microsoft.com/en-us/platform/support/policy>
- .NET 8 ve 9 destek sonu duyurusu — <https://devblogs.microsoft.com/dotnet/dotnet-8-9-end-of-support/>

Standartlar

- HTTP semantiği — RFC 9110
- HTTP/2 — RFC 7540 · HTTP/3 — RFC 9114